Token-Level Backtracking for Corrective Reasoning in LLMs

Siddharth Mohanty*
University of California, Santa Cruz

April 26, 2026

Abstract

Modern Large Language Models (LLMs) generate text autoregressively, meaning each token depends irrevocably on all previous tokens. Once a mistaken inference occurs, the future reasoning chain becomes polluted, often leading to hallucinations or collapsed logic. Existing methods address this retrospectively rather than dynamically at generation time. We propose Token-Level Backtracking, a lightweight, inference-time mechanism that detects reasoning errors via unsupervised signals and executes a semantic rollback to an earlier "weak node." By gating rollbacks behind a speculative divergence test, our method ensures computational efficiency, allocating compute only when alternative reasoning trajectories demonstrate significant semantic deviation. In evaluation, LLMs with backtracking and rollback consistently outperformed standard generation by 4–9% over benchmarks including GSM8K and HumanEval, while retaining 88% of baseline throughput.

1 Introduction

Modern LLMs generate text autoregressively: each token depends irrevocably on all previous tokens [12]. This means that once a model makes a mistaken inference, the entire future reasoning chain becomes "polluted," and the model is forced to justify or elaborate on a false premise (the "sunk cost fallacy" of generation).

Existing methods like beam search, self-consistency [11], or reflection [8, 10] improve results *after* mistakes occur, but they don't directly address the underlying issue: there is no mechanism to rewind and repair reasoning at the moment an error appears. Recent inference-time reasoning models such as OpenAI o1 [9] and DeepSeek-R1 [4] demonstrate that extended chain-of-thought generation improves performance, and notably, DeepSeek-R1-Zero organically learned to emit self-correction tokens ("Wait...") mid-generation under reinforcement learning—suggesting that models *want* to backtrack but lack a principled mechanism for doing so.

In this work, we treat the token stream not as a rigid list, but as a dynamic tree of reasoning [13]. Some tokens in the tree act as "polluting nodes"—points where reasoning becomes incorrect or contradictory. Instead of pushing more context to override a mistake, we delete tokens back to the point of error and resample a new continuation. This creates a form of local backtracking (or "System 2" self-correction [6]) inside the "System 1" autoregressive generation process.

Unlike speculative decoding [1, 7]—which uses a draft model to accelerate inference while preserving output distribution—our method repurposes the draft model as a *semantic divergence detector*, triggering rollbacks only when an alternative reasoning branch is substantively different. Unlike lookahead decoding [5], which parallelizes token generation without semantic awareness, our rollbacks are conditioned on detected reasoning errors. The result is a targeted, compute-efficient correction mechanism that requires no retraining and is model-agnostic.

Our main contributions are as follows:

1. We introduce a suite of unsupervised signals—trigger tokens, entropy spikes, weak-node analysis, and likelihood contrast tests—for detecting reasoning errors at inference time without any supervised labels or fine-tuning.

2. We present a speculative-gated rollback algorithm that leverages a lightweight draft model to perform semantic divergence testing, ensuring that computational overhead is incurred only when an alternative reasoning trajectory is substantively different.

*smohant5@ucsc.edu

3. We demonstrate consistent improvements of 4–9% accuracy on GSM8K and 3–7% on HumanEval over standard autoregressive baselines, with minimal latency overhead (<12% wall-clock increase).

2 Related Work

Inference-Time Search and Reasoning. Chain-of-thought prompting [12] demonstrated that eliciting intermediate reasoning steps substantially improves LLM performance on multi-step tasks. Tree of Thoughts [13] generalized this by exploring multiple reasoning branches via breadth-first or depth-first search, though at significant computational cost. More recently, inference-time reasoning models such as OpenAI o1 [9] and DeepSeek-R1 [4] have shown that extended internal deliberation during generation yields state-of-the-art results on mathematical and scientific benchmarks. Monte Carlo Tree Search (MCTS) has also been applied to LLM reasoning, treating each generation step as a node in a search tree and using value estimates to guide expansion. Our work shares the motivation of exploring multiple reasoning paths but differs in that we do not require exhaustive search; instead, we selectively backtrack only at detected error points.

Speculative Decoding. Speculative decoding [1, 7] uses a smaller draft model to propose candidate token sequences that are then verified by the target model, accelerating inference while provably preserving the output distribution. Our method repurposes this draft-verify paradigm for a fundamentally different goal: rather than using the draft model to speed up token acceptance, we use it as a *semantic probe* to evaluate whether an alternative reasoning branch diverges meaningfully from the current trajectory. This reframing transforms speculative decoding from a latency optimization into a reasoning quality mechanism.

Self-Correction and Reflection. Self-Refine [8] iteratively prompts the model to critique and revise its own output, while Reflexion [10] maintains an episodic memory of past failures to improve subsequent attempts. Both operate *post hoc*—the model must first complete (or substantially advance) a flawed generation before correction begins. In contrast, our approach intervenes *during* generation at the precise token where reasoning degrades, preventing error propagation rather than repairing it after the fact.

Token-Level Resampling and Lookahead Methods. Resample-Previous-Tokens (RPT) proposes resampling earlier tokens conditioned on future context to improve coherence. Lookahead decoding [5] parallelizes n-gram generation using Jacobi iteration to reduce wall-clock latency. Both methods operate without semantic awareness of reasoning quality. Our approach is more targeted: resampling is triggered only at semantically identified error points and gated by a divergence test, ensuring that rollbacks are both necessary and productive.

3 Methodology

3.1 Unsupervised Detection of Polluting Nodes

We detect potential reasoning errors using a combination of semantic and statistical unsupervised signals:

- Trigger Tokens: Explicit linguistic markers (e.g., "Wait", "Actually", "But that means...") indicating the model is about to contradict itself.
- Entropy Spikes: Sudden jumps in perplexity, suggesting the model is "forcing" a justification for a previous weak step.
- "Weak Node" Analysis: Instances where the probability difference between the top two tokens is negligible ($< \epsilon$). These indices are marked as "uncertainty checkpoints"; if a contradiction occurs later, the model knows exactly where the error likely started.
- Likelihood Contrast Tests: Checking whether the model, conditioned on the "Wait" context, would confidently refuse to regenerate the earlier claim.
- Self-Consistency: Resampling multiple short continuations to see if they diverge or reproduce the suspect statement.

3.2 Dynamic Checkpoint and Resampling Strategy

We maintain a buffer of variable-sized checkpoints (previous clause, sentence, or equation start). When a polluting node is detected, we execute a Semantic Rollback:

1. Identify the Inception Point: Do not just delete the trigger token; rewind to the nearest "Weak Node" or the start of the current reasoning block to address the lag between making an error and realizing it.
2. Speculative Divergence Testing: Before committing to a full rollback (which discards the current path's KV cache), we utilize Speculative Decoding with a lightweight draft model to rapidly generate a short lookahead on the alternative branch. We then compute the semantic distance between this draft continuation and the original "polluted" trajectory. A rollback is only executed if this test reveals a significant semantic difference (e.g., a fundamentally different logical step or mathematical operation). If the speculative branch merely yields a trivial rewording, the rollback is aborted to conserve compute.
3. Contrastive Resampling: Once semantic divergence is confirmed, resample from the checkpoint with negative constraints. Apply a massive logit penalty to the specific sequence that led to the error, forcing the model to explore the fundamentally different reasoning branch identified by the draft model.
4. Tree-Based Optimization: Instead of clearing the KV cache immediately, mark the "bad branch" as inactive. This allows for "undoing the undo" if the new path proves to be even lower probability.
5. Loop Guardrails: Enforce a `Max_Backtracks` limit per reasoning step to prevent "apology loops" (where the model repeatedly errors and rewinds).

4 Experimental Setup

4.1 Models and Hardware

To validate the model-agnostic nature of our approach, we evaluate Token-Level Backtracking across four diverse model families spanning a range of architectures and parameter scales:

- **Llama-3.1**: We use `meta/llama-3.1-70b-instruct` as the primary generator paired with `meta/llama-3.1-8b-instruct` as the draft model.
- **Mistral**: We use `Mistral-Large-2` (123B) as the primary generator paired with `Mistral-7B-Instruct-v0.3` as the draft model.
- **Qwen-2.5**: We use `Qwen2.5-72B-Instruct` as the primary generator paired with `Qwen2.5-7B-Instruct` as the draft model.
- **Gemma-2**: We use `gemma-2-27b-it` as the primary generator paired with `gemma-2-2b-it` as the draft model, testing whether the method scales down to smaller model pairs.

All experiments are conducted on a cluster of 4× NVIDIA H100 80GB GPUs using CUDA with 4-bit quantization (`BitsAndBytes`) and `bfloat16` precision to maximize memory efficiency. For the 123B Mistral-Large-2 configuration, we employ tensor parallelism across 2 GPUs.

4.2 Benchmarks and Evaluation

We assess our method across four benchmarks spanning mathematical reasoning, code generation, and general multi-step logic:

- **GSM8K** [3]: 1,319 grade-school math word problems requiring multi-step arithmetic reasoning. We report exact-match accuracy on the test split.
- **MATH**: 5,000 competition-level mathematics problems across seven categories (algebra, number theory, geometry, etc.). We report accuracy on the 500-problem test split using the `minerva` answer extraction protocol.

- **HumanEval** [2]: 164 Python programming problems. We report pass@1 using greedy decoding and pass@5 with temperature sampling (T=0.8).
- **AIME 2024**: 30 problems from the American Invitational Mathematics Examination, requiring advanced high-school level reasoning with integer answers in $[0, 999]$.

All benchmarks use a few-shot chain-of-thought prompt template that requires step-by-step reasoning before the final answer. Our evaluation script parses the generated text and compares the extracted answer against the gold standard.

4.3 Implementation Details

To determine semantic divergence during the speculative rollback phase, we compute the cosine distance ($1 - \text{cosine_similarity}$) between the representations of the generated branches. A rollback is only triggered if this distance exceeds a configurable threshold ($\tau = 0.35$). When resampling after a rollback, we apply a significant logit penalty (+30.0) to the incorrect path to force exploration of a new reasoning trajectory. We enforce a maximum limit of 3 backtracks per reasoning step and 50 total backtracks per sequence to prevent infinite loops. We compare our backtracking method against a standard autoregressive baseline generator using identical model weights and temperature sampling ($T = 0.8, p = 0.9$).

5 Results

5.1 Main Results

Table 1 summarizes performance across all four benchmarks using the Llama-3.1-70B configuration. Token-Level Backtracking yields consistent improvements, with the largest gains on multi-step arithmetic chains where a single upstream error typically cascades through subsequent steps.

Table 1: Accuracy (%) comparison across benchmarks using Llama-3.1-70B. Tokens/s measured on a single H100 GPU.

Method	GSM8K	MATH	HumanEval p@1	AIME '24	Tok/s
Standard Autoregressive	74.2	41.6	61.0	13.3	42.7
+ Self-Consistency (k=5)	78.6	45.2	63.4	16.7	9.1
+ Self-Refine (1 iter)	76.9	43.8	64.8	16.7	21.3
+ Tree of Thoughts (b=3)	79.1	46.0	62.1	20.0	6.4
+ Backtracking (Ours)	**82.1**	**48.4**	**67.7**	**23.3**	**37.8**

Our method retains ∼88% of the baseline throughput (37.8 vs. 42.7 tokens/s), since the speculative divergence gate prevents unnecessary rollbacks on ∼73% of candidate trigger points. The improvement is most pronounced on AIME, where problems require 10–15 chained reasoning steps and a single arithmetic slip typically invalidates the entire solution. Self-consistency and Tree of Thoughts achieve competitive accuracy but at a 4.7–6.7× throughput penalty due to full-sequence resampling.

5.2 Cross-Model Generalization

A key claim of our work is model-agnosticism: because Token-Level Backtracking operates solely on output logits and hidden-state cosine distances, it should transfer across architectures without modification. Table 2 validates this across four model families.

Several trends emerge. First, backtracking provides consistent gains (Δ = 5.2–7.9) regardless of base model strength, suggesting that the method addresses a fundamental limitation of autoregressive decoding rather than compensating for model-specific weaknesses. Second, the smaller Gemma-2 pair (27B/2B) achieves proportionally the largest improvement on MATH (+6.5 points), indicating that weaker models benefit *more* from mid-generation error correction, as they produce reasoning errors at a higher base rate. Third, the strongest base model (Qwen-2.5-72B at 82.4% GSM8K) still gains 5.2 points, demonstrating that backtracking is not merely a crutch for weaker models.

Table 2: GSM8K and MATH accuracy (%) across model families. Δ denotes absolute improvement over each model's autoregressive baseline.

Model Family	Params	GSM8K Base	GSM8K +BT	Δ	MATH Base	MATH +BT
Llama-3.1	70B / 8B	74.2	82.1	+7.9	41.6	48.4
Mistral-Large-2	123B / 7B	79.8	85.3	+5.5	47.2	53.1
Qwen-2.5	72B / 7B	82.4	87.6	+5.2	49.8	55.4
Gemma-2	27B / 2B	68.1	74.9	+6.8	34.2	40.7

5.3 Scaling Behavior and Draft Model Size

We investigate how the ratio between the primary generator and the draft model affects both accuracy and throughput. Using Llama-3.1-70B as the fixed primary model, we vary the draft model across three sizes (Table 3).

Table 3: Effect of draft model size on GSM8K performance. Primary model: Llama-3.1-70B.

Draft Model	Params	Acc. (%)	Tok/s	Avg. BT/Seq	Gate Precision
None (baseline)	—	74.2	42.7	—	—
Llama-3.2-1B	1B	78.9	40.1	3.8	0.61
Llama-3.1-8B	8B	82.1	37.8	2.4	0.79
Llama-3.1-70B (self)	70B	83.0	22.4	1.9	0.86

The 8B draft model occupies a favorable point on the Pareto frontier: it captures 91% of the accuracy gain of self-drafting (82.1 vs. 83.0) while retaining 89% of baseline throughput (37.8 vs. 42.7 tok/s). The 1B draft model is too weak to reliably distinguish semantic divergence from noise, yielding a gate precision of only 0.61 (i.e., 39% of triggered rollbacks were unnecessary). Self-drafting with the full 70B model achieves the highest accuracy but at a prohibitive 1.9× latency increase, making it impractical for deployment.

5.4 HumanEval and Code Generation Analysis

Table 4 presents detailed HumanEval results across model families at both pass@1 (greedy) and pass@5 (T=0.8).

Table 4: HumanEval pass@k (%) across model families.

Model	p@1 Base	p@1 +BT	p@5 Base	p@5 +BT
Llama-3.1-70B	61.0	67.7	78.4	83.1
Mistral-Large-2	64.6	69.5	81.2	85.0
Qwen-2.5-72B	71.3	76.2	85.7	89.4
Gemma-2-27B	51.8	58.5	69.3	76.1

Backtracking improves pass@1 by 4.9–6.8 points across all families. Qualitatively, we observe that the most common rollback triggers in code generation are (1) incorrect loop bounds that would cause off-by-one errors, (2) type mismatches caught by entropy spikes when the model struggles to continue after an ill-typed expression, and (3) logical inversions in conditional branches. The pass@5 improvements are smaller in absolute terms (+3.7–6.8) but still significant, suggesting that backtracking provides orthogonal benefits to sample-level diversity.

5.5 Ablation Study

To isolate the contribution of each component, we ablate the speculative divergence gate and the contrastive resampling penalty on GSM8K (Table 5).

Removing the speculative divergence gate causes the system to execute ∼3× more backtracks with diminishing returns: many rollbacks target surface-level rephrasings rather than genuine reasoning errors, reducing accuracy by 2.8 points and increasing latency by 31%. Removing the contrastive logit penalty leads to a 2.1-point drop as the model occasionally re-derives the same erroneous reasoning path. Weak node analysis contributes 3.3 points by

Table 5: Ablation on GSM8K. Each row removes one component from the full system.

Configuration	Acc. (%)	Avg. Backtracks/Seq
Full System	82.1	2.4
− Speculative Divergence Gate	79.3	7.1
− Contrastive Logit Penalty	80.0	2.6
− Weak Node Analysis	78.8	3.9
Baseline (no backtracking)	74.2	—

enabling precise rollback targeting; without it, rollbacks default to fixed-window heuristics that often overshoot or undershoot the true error locus.

5.6 Sensitivity to Divergence Threshold τ

The semantic divergence threshold τ controls the trade-off between rollback frequency and precision. Table 6 reports GSM8K accuracy and throughput as τ varies from aggressive (0.15) to conservative (0.55).

Table 6: Sensitivity to divergence threshold τ on GSM8K (Llama-3.1-70B / 8B).

τ	Acc. (%)	Tok/s	BT/Seq	Gate Prec.
0.15	79.6	31.2	5.7	0.52
0.25	81.3	35.4	3.4	0.68
0.35	**82.1**	**37.8**	**2.4**	**0.79**
0.45	80.4	39.8	1.3	0.88
0.55	77.1	41.3	0.6	0.93

At low τ, the system triggers too aggressively, wasting compute on cosmetic rollbacks and occasionally discarding correct reasoning paths (gate precision 0.52). At high τ, too few genuine errors are caught—at $\tau = 0.55$ the system averages only 0.6 backtracks per sequence, barely distinguishable from the baseline. The optimal $\tau = 0.35$ balances rollback precision (0.79) with sufficient coverage to catch multi-step errors.

5.7 Qualitative Error Analysis

To understand *what* backtracking corrects, we manually categorize 200 GSM8K examples where our method changed the final answer (relative to the baseline) using Llama-3.1-70B. Of these, 156 (78%) flipped an incorrect baseline answer to correct, 31 (15.5%) maintained a correct answer through an alternative reasoning path, and 13 (6.5%) introduced a new error (i.e., the baseline was correct but backtracking degraded the answer). Among the 156 successful corrections, the most common error categories were:

- **Arithmetic errors** (41%): Incorrect multiplication, division, or carry propagation in intermediate steps.
- **Variable misattribution** (23%): Swapping or confusing quantities from the problem statement (e.g., using Alice's rate where Bob's was needed).
- **Logical inversions** (18%): Subtracting instead of adding, or inverting an inequality direction.
- **Premature conclusion** (12%): Skipping a required step and outputting an intermediate result as the final answer.
- **Unit/conversion errors** (6%): Failing to convert between units (e.g., hours to minutes) before a calculation.

The 13 cases where backtracking introduced errors predominantly involved problems where the baseline reached the correct answer through a "lucky" cancellation of two opposing errors—backtracking corrected the first error but exposed the second, yielding an incorrect final result. This suggests a natural ceiling for single-pass backtracking and motivates future work on recursive or multi-pass correction.

6 Discussion and Applications

This approach breaks the rigid left-to-right dependency chain of autoregressive decoding. It creates a lightweight, inference-time mechanism for internal error correction, enabling LLMs to:

- Fix "reasoning collapse": Prevent one bad arithmetic step from ruining a long proof.
- Repair logical inconsistencies in real-time.
- Compute-Efficient Correction: By gating rollbacks behind a speculative semantic divergence check, we avoid wasting FLOPs on trivial rephrasings.
- Avoid "hallucination lock-in" by catching errors early.
- Outperform naïve decoding without the need for retraining or fine-tuning.

The method complements recent work like Resample-Previous-Tokens (RPT) and lookahead decoding, but is more targeted: it resamples only at points where the model detects its own wrongness and confirms an alternative semantic path.

This engine is highly applicable to domains requiring strict logical coherence:

- Mathematical Reasoning: (GSM8K, ProofWriter) Catching upstream errors immediately.
- Code Generation: Fixing incorrect logic or syntax before the function is closed.
- Agentic Models: Agents can fix their reasoning before irrevocable tool calls.
- Safety: Preventing runaway hallucinations or toxic spirals early in generation.

7 Conclusion

We presented Token-Level Backtracking, an inference-time mechanism that equips autoregressive models with a principled "Delete Key"—the ability to detect reasoning errors via unsupervised signals, verify that an alternative path is semantically meaningful through speculative divergence testing, and resample from a targeted checkpoint with contrastive constraints. On GSM8K and HumanEval, our method improves accuracy by 4–9% over standard decoding while retaining 88% of baseline throughput, substantially outperforming post-hoc correction methods in efficiency.

The approach is model-agnostic, requires no fine-tuning or retraining, and leverages only the model's own probability distributions—mimicking how humans revise their thinking mid-stream rather than restarting from scratch. Future work will explore adaptive draft model sizing, where the capacity of the speculative model scales with the detected complexity of the reasoning error, as well as extending the framework to multi-turn agentic settings where irrevocable tool calls demand even higher precision in pre-action reasoning verification.

References

[1] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[4] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[5] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of LLM inference using lookahead decoding. In *Proceedings of the 41st International Conference on Machine Learning*, pages 14060–14079. PMLR, 2024.

[6] Daniel Kahneman. *Thinking, Fast and Slow*. Macmillan, 2011.

[7] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[8] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.

[9] OpenAI. OpenAI o1 technical report. `https://openai.com/index/learning-to-reason-with-llms/`, 2024.

[10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

[11] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[12] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[13] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.